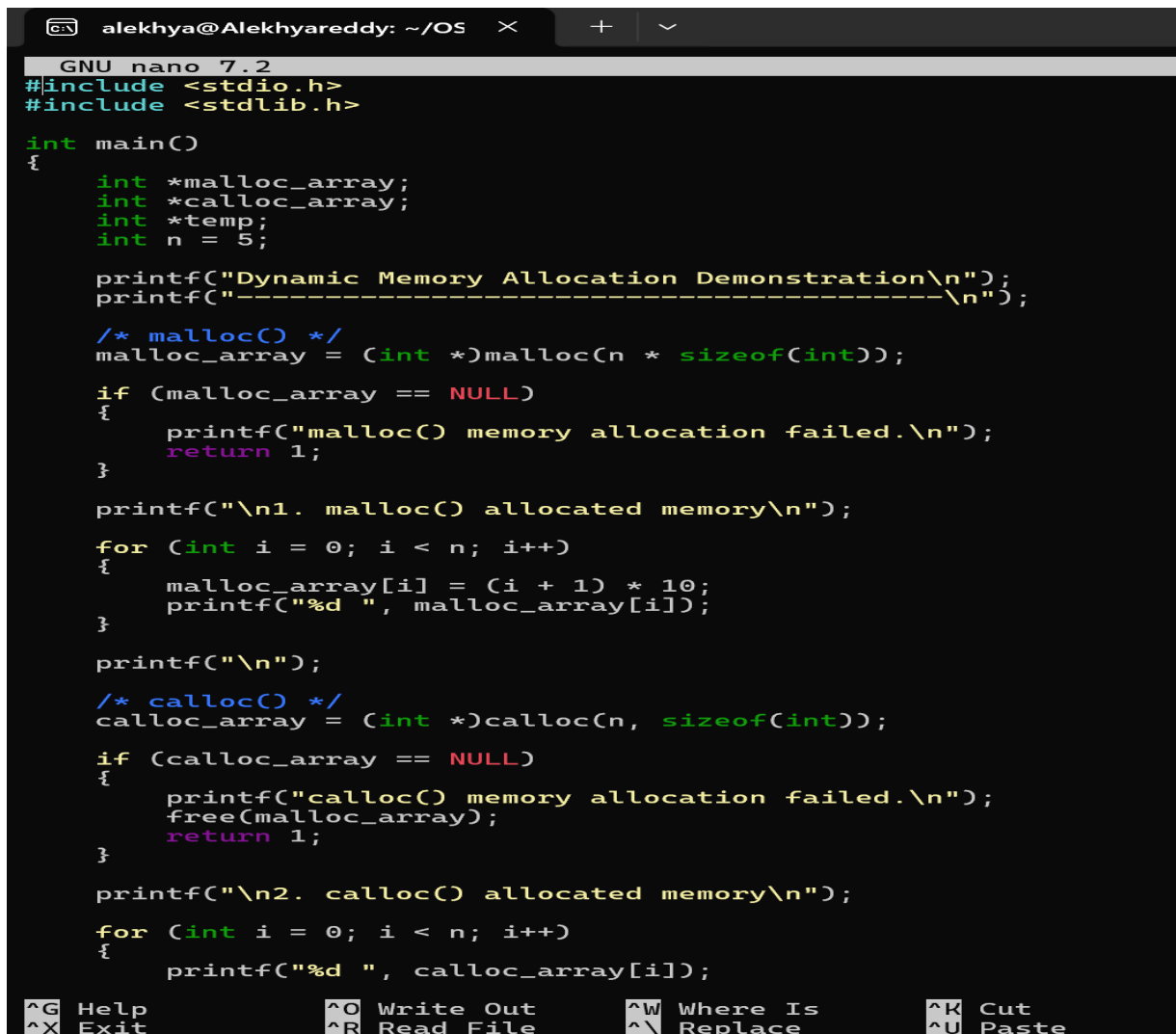


WEEK 8 PRACTICAL

1: Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(). Also identify and analyze memory leaks using Valgrind.

Aim : To understand dynamic memory allocation in C using malloc(), calloc(), realloc(), and free(), and to detect memory leaks using Valgrind.

Program :



```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *malloc_array;
    int *calloc_array;
    int *temp;
    int n = 5;

    printf("Dynamic Memory Allocation Demonstration\n");
    printf("-----\n");

    /* malloc() */
    malloc_array = (int *)malloc(n * sizeof(int));
    if (malloc_array == NULL)
    {
        printf("malloc() memory allocation failed.\n");
        return 1;
    }

    printf("\n1. malloc() allocated memory\n");
    for (int i = 0; i < n; i++)
    {
        malloc_array[i] = (i + 1) * 10;
        printf("%d ", malloc_array[i]);
    }

    printf("\n");

    /* calloc() */
    calloc_array = (int *)calloc(n, sizeof(int));
    if (calloc_array == NULL)
    {
        printf("calloc() memory allocation failed.\n");
        free(malloc_array);
        return 1;
    }

    printf("\n2. calloc() allocated memory\n");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", calloc_array[i]);
    }
}
```



alekhya@Alekhyareddy: ~/OS



GNU nano 7.2

```
{
    printf("%d ", calloc_array[i]);
}

printf("\n");

/* realloc() */
temp = (int *)realloc(malloc_array, 10 * sizeof(int));

if (temp == NULL)
{
    printf("\nrealloc() memory allocation failed.\n");
    free(malloc_array);
    free(calloc_array);
    return 1;
}

malloc_array = temp;

printf("\n3. realloc() increased memory size\n");

for (int i = 5; i < 10; i++)
{
    malloc_array[i] = (i + 1) * 10;
}

for (int i = 0; i < 10; i++)
{
    printf("%d ", malloc_array[i]);
}

printf("\n");

/* free() */
free(malloc_array);
free(calloc_array);

malloc_array = NULL;
calloc_array = NULL;

printf("\n4. free() released allocated memory\n");

printf("\nDynamic memory allocation completed successfully.\n");

return 0;
}
```

Help
 Exit

Write Out
 Read File

Where Is
 Replace

Cut
 Paste

Exchange
 Justify



alekhya@Alekhyaireddy: ~/OS



GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *data;

    data = (int *)malloc(5 * sizeof(int));

    if (data == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        data[i] = i + 1;
    }

    printf("Allocated data: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    /*
     * Memory leak:
     * free(data) is intentionally omitted.
     */

    return 0;
}
```

Output :

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano dynamic_memory.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra -g dynamic_memory.c -o dynamic_memory
alekhya@Alekhyareddy:~/OSSP$ ./dynamic_memory
Dynamic Memory Allocation Demonstration
-----

1. malloc() allocated memory
10 20 30 40 50

2. calloc() allocated memory
0 0 0 0 0

3. realloc() increased memory size
10 20 30 40 50 60 70 80 90 100

4. free() released allocated memory

Dynamic memory allocation completed successfully.

alekhya@Alekhyareddy:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./dynamic_memory
==744== Memcheck, a memory error detector
==744== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==744== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==744== Command: ./dynamic_memory
==744==
Dynamic Memory Allocation Demonstration
-----

1. malloc() allocated memory
10 20 30 40 50

2. calloc() allocated memory
0 0 0 0 0

3. realloc() increased memory size
10 20 30 40 50 60 70 80 90 100

4. free() released allocated memory

Dynamic memory allocation completed successfully.
==744==
==744== HEAP SUMMARY:
==744==   in use at exit: 0 bytes in 0 blocks
==744==   total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==744==
==744== All heap blocks were freed -- no leaks are possible
==744==
==744== For lists of detected and suppressed errors, rerun with: -s
==744== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
alekhya@Alekhyareddy:~/OSSP$ ^C
alekhya@Alekhyareddy:~/OSSP$ nano memory_leak.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra -g memory_leak.c -o memory_leak
alekhya@Alekhyareddy:~/OSSP$ ./memory_leak
Allocated data: 1 2 3 4 5
```

```
alekhya@Alekhyareddy:~/OSSP$ valgrind --leak-check=full --show-leak-kinds=all ./memory_leak
==756== Memcheck, a memory error detector
==756== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==756== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==756== Command: ./memory_leak
==756==
Allocated data: 1 2 3 4 5
==756==
==756== HEAP SUMMARY:
==756==   in use at exit: 20 bytes in 1 blocks
==756==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==756==
==756== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==756==   at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==756==   by 0x1091BE: main (memory_leak.c:8)
==756==
==756== LEAK SUMMARY:
==756==   definitely lost: 20 bytes in 1 blocks
==756==   indirectly lost: 0 bytes in 0 blocks
==756==   possibly lost: 0 bytes in 0 blocks
==756==   still reachable: 0 bytes in 0 blocks
==756==   suppressed: 0 bytes in 0 blocks
==756==
==756== For lists of detected and suppressed errors, rerun with: -s
==756== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
alekhya@Alekhyareddy:~/OSSP$ |
```

2: Write a C program to demonstrate the Copy-on-Write (COW) mechanism after a fork() system call. Show that the parent and child processes have separate copies of the data when the child modifies it.

Aim : To demonstrate how Copy-on-Write (COW) works after fork(), and observe that the parent and child initially share memory pages. When the child modifies the data, the operating system creates a separate copy of the modified page for the child.

Program :

```
alekhya@Alekhyareddy: ~/OS  ×  +  v
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int data = 100;

    printf("Copy-on-Write (COW) Demonstration\n");
    printf("-----\n");

    printf("Before fork():\n");
    printf("Parent PID: %d\n", getpid());
    printf("Data      : %d\n\n", data);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        /* Child process */
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Data before modification: %d\n", data);

        printf("Child modifying data...\n");
        data = 200;

        printf("Data after modification: %d\n", data);
        printf("Child has its own copy of the data.\n");

        exit(0);
    }
    else
    {
        /* Parent process */
        wait(NULL);

        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Data after child modification: %d\n", data);
        printf("Parent data remains unchanged.\n");
    }

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify
```

Output :

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano cow_practical.c
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra cow_practical.c -o cow_practical
alekhya@Alekhyareddy:~/OSSP$ ./cow_practical
Copy-on-Write (COW) Demonstration
-----
Before fork():
Parent PID: 822
Data      : 100

Child Process
Child PID: 823
Data before modification: 100
Child modifying data...
Data after modification: 200
Child has its own copy of the data.

Parent Process
Parent PID: 822
Data after child modification: 100
Parent data remains unchanged.
alekhya@Alekhyareddy:~/OSSP$ ps -ef | grep cow_practical
alekhya      825      788  0 11:33 pts/0    00:00:00 grep --color=auto cow_practical
```